MANUAL TÉCNICO PORTAL DE PROVEEDORES

"Conexión entre Base de Datos, Programación e Interfaz de Usuario"

Área de Sistemas
Logiran S.A.

Índice

1. Esquema y Organización de la Base de Datos 3
2. Relaciones y Estructuras de la Base de datos 3
2.1. Tabla <consentimiento_transportespot> 3
3. Aplicación del Patrón MVC en el Desarrollo del Portal de Proveedores 4
3.1. Modelo 4
3.1.1. Características del Modelo 4
3.2. Controlador 5
3.2.1. Características del Controlador 5
3.3. Vista 6
3.3.1. Características de la Vista 6
3.4. Flujo General del MVC 7

1. Esquema y Organización de la Base de Datos

Las aplicaciones web están basadas en el modelo Cliente Servidor, en el cual se distribuyen las tareas entre los proveedores de recursos o servicios, llamados "Servidores" y los llamados "Clientes" que son conectados por (browser o navegadores) y utilizados por usuarios. En la siguiente imagen se muestra la arquitectura para consultar los diferentes servicios de la aplicación.

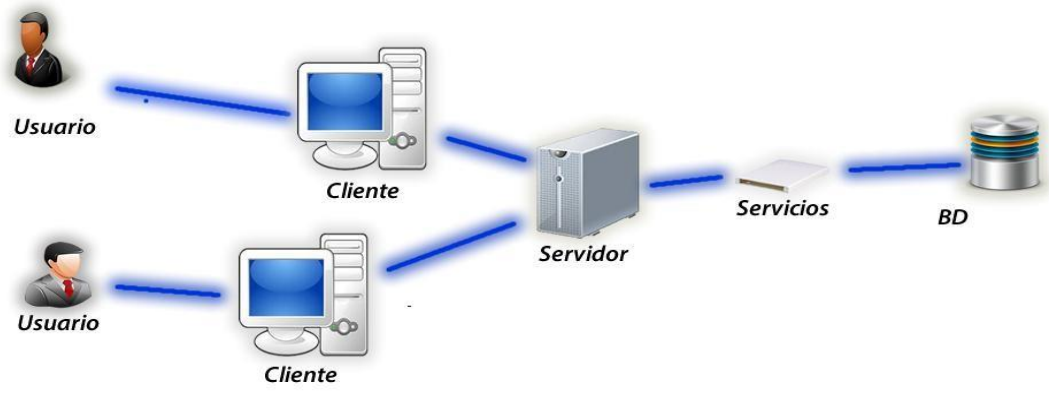

Ilustración 1. Modelo Cliente-Servidor

2. Relaciones y Estructuras de la Base de datos

2.1. Tabla <consentimiento_transportespot>

La tabla **IT.consentimiento_transportespot** almacena la información y los documentos de consentimiento de los transportistas.

- El campo **razon_social** almacena el nombre de la razón social o empresa del transportista.
- El campo **cedula** almacena el número de identificación del transportista o del conductor.
- El campo **conductor** almacena el nombre del conductor.
- El campo **tipo_licencia** almacena el tipo de licencia de conducir que posee el conductor.
- El campo **celular** almacena el número de teléfono celular de contacto del transportista o conductor.
- El campo **tipounidad** almacena el tipo de unidad de transporte que utiliza (por ejemplo, camión, furgoneta, etc.).
- El campo **adj_licencia** almacena la ruta o URL del archivo adjunto correspondiente a la licencia de conducir.
- El campo **adj_cedula** almacena la ruta o URL del archivo adjunto correspondiente a la cédula de identidad.
- El campo **adj_lmatricula** almacena la ruta o URL del archivo adjunto correspondiente a la matrícula del vehículo.

- El campo **adj_poperaciones** almacena la ruta o URL del archivo adjunto correspondiente al permiso de operaciones.

consentimiento_transportespot	
123 id_spot	int NOT NULL
fecha_registro	datetime
A-Z razon_social	text
A-Z cedula	varchar(15)
A-Z conductor	varchar(200)
A-Z tipo_licencia	varchar(1)
A-Z adj_licencia	text
A-Z adj_cedula	text
A-Z adj_lmatricula	text
A-Z adj_poperaciones	text
A-Z celular	varchar(15)
A-Z tipounidad	varchar(100)

Ilustración 2. Entidades de tabla consentimiento_transportespot

3. Aplicación del Patrón MVC en el Desarrollo del Portal de Proveedores

Este proyecto implementa el patrón de diseño *Modelo-Vista-Controlador (MVC)*, que organiza el código en tres componentes principales: Modelo, Vista y Controlador.

3.1. Modelo

El **modelo** es responsable de la lógica de negocio y la interacción con la base de datos. En este proyecto, los modelos se encuentran en la ruta:

proveedorspot\Model\

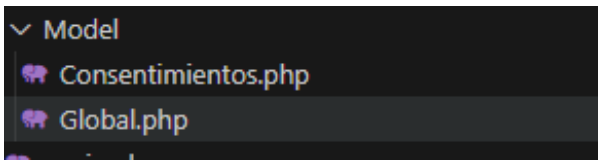

Ilustración 3. Modelos del portal

3.1.1.Características del Modelo

- **Conexión a la base de datos.** El modelo utiliza una función como conexionSQL() para establecer la conexión con la base de datos (SQL Server). Donde cada función en el modelo realiza una consulta específica a la base de datos. Por ejemplo:
  - Consentimientos(): Almacena datos en la tabla consentimiento_transportespot.
- **Estructura de datos.** Los datos obtenidos de la base de datos se procesan para luego ser almacenados en un array, que posteriormente se devuelven al controlador.

```php
public function registrar($data): void
{
    $Global  = new ModelGlobal();
    $conn      = conexionSQL();

   $imagenes = explode(separator: "|*",string: $data["url_imagen"]);

$sqlx = "INSERT INTO  [IT].[consentimiento_transportespot]
            ([fecha_registro]
            ,[razon_social]
            ,[cedula]
            ,[conductor]
            ,[tipo_licencia]
            ,[celular]
            ,[tipounidad]
            ,[adj_licencia]
            ,[adj_cedula]
            ,[adj_lmatricula]
            ,[adj_poperaciones])
VALUES(?,?,?,?,?,?,?,?,?,?,?)";
    $paramsx = array(date(format: 'Y-m-d H:i:s'),$data['razonsocial'],$data['cedula'], $data['conductor_name'],$data['tipo_lic'],
    $data['celular'],$data['tipounidad'],
    $imagenes[0],$imagenes[1],$imagenes[2],$imagenes[3]);

    //print_r($paramsx);
$stmtx = sqlsrv_prepare(conn: $conn, sql: $sqlx, params: $paramsx);
if (sqlsrv_execute( stmt: $stmtx ) === false) {
    $Global->salir_json(n: 2,error: "ERROR EN LA BASE DE DATOS CONSULTA " . $sqlx);

} else {
    $Global->salir_json(n: 1,error: "");
}
```

Ilustración 4. Función de ejemplo de uno de los modelos del proyecto

3.2. Controlador

El controlador actúa como intermediario entre el modelo y la vista. En este proyecto, los controladores están en la siguiente ruta:

proveedorspot\Controller\

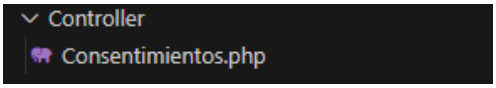

Ilustración 5. Controladores del Proyecto

3.2.1. Características del Controlador

- **Procesamiento de solicitudes.** El controlador recibe las solicitudes (generalmente POST o GET) y decide qué acción realizar en función de los parámetros enviados.
- **Llamadas al modelo.** El controlador llama a las funciones del modelo para obtener o manipular datos. Por ejemplo:
    - Registrar() llama a ModelCliente->registrar() para almacenar datos en la base de datos.
- **Devolución de datos.** Los datos obtenidos del modelo se procesan y se devuelven al cliente (generalmente en formato JSON) para ser consumidos por el frontend.

```php
function registrar(): void
{
    //$Global     = new ModelGlobal();
    $Consult    = new ModelCliente();

    $data["tipo_lic"]               = $_POST['tipo_lic'];
    $data["conductor_name"]         = $_POST['conductor_name'];
    $data["cedula"]         = $_POST['cedula'];
    $data["celular"]            = $_POST['celular'];
    $data["tipounidad"]         = $_POST['tipounidad'];
    $data["url_imagen"]         = $_POST['url_imagen'];
    $data["razonsocial"]                        = $_POST["razonsocial"];
    $i                                      = 1;
    $movements  = $Consult->registrar(data: $data);
    foreach ($movements as $movement) {

        $dato["result"]    = $movement["result"];
        $dato["error"]     = $movement["error"];
    }
    echo json_encode(value: $dato);
}

?>
```

Ilustración 6. Ejemplo de función que maneja la lógica dentro de controlador

3.3. Vista

La vista es responsable de la interfaz de usuario. En este proyecto, las vistas están implementadas principalmente en archivos JavaScript y HTML y se encuentran en el archivo Index.php.

Ilustración 7. Interfaz gráfica de la vista principal

3.3.1. Características de la Vista

- **Interacción con el usuario.** Formularios HTML permiten la entrada de datos por parte del usuario.
- **Solicitudes al controlador.** Utiliza AJAX (jQuery) para enviar datos al backend y recibir respuestas.
- **Renderización de datos.** Muestra mensajes de éxito/error y actualiza la interfaz según la respuesta del backend.

3.4. Flujo General del MVC

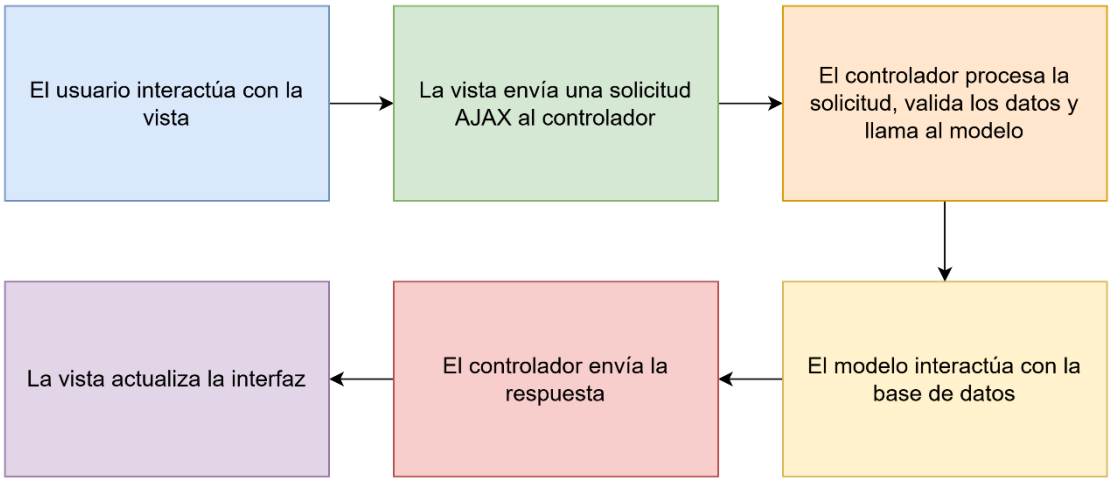

Ilustración 8. Flujo del portal de Proveedores